System and Device Programming
On Off Exam
25.06.2024

Ex 1 (1.5 points)

Suppose to run the following program using the command:
`./pgrm 3`
Indicate how many times the program displays a PID. Note that wrong answers imply a penalty in the final score.

```
#define N 100

int main (int argc, char **argv){
  int n;
  char str[N];
  setbuf (stdout, 0);
  printf ("%d ", getpid());
  n = atoi(argv[1]);
  if (n<=0)
    return 1;
  if (fork()==0) {
    sprintf (str, "%d", n-1);
    execlp (argv[0], argv[0], str, NULL);
  }
  sprintf (str, "%s %d", argv[0], n-1);
  system (str);
}
```

Choose one or more options:

1. ⬜ 7
2. ⬜ 9
3. ⬜ 11
4. ⬜ 13
5. ⬜ 15
6. ⬜ 16
7. ⬜ 17

Ex 2 (1.5 points)

Suppose to run the following program. Indicate the possible admissible outputs. Note that wrong answers imply a penalty in the final score.

```
std::map<int, int> f (const std::vector<int>& vec, std::set<int> s) {
    std::map<int, int> m;
    for (const int &e : vec) {
      auto it = s.find(e);
      if (it != s.end())
        ++m[e];
    }
    return m;
}

int main() {
  std::vector<int> vec = {1, 2, 6, 2, 3, 3, 2, 3, 4, 5, 4, 4, 4, 5, 5, 5, 6, 6};
```

```
  std::set<int> s = {2, 4, 6};
  std::map<int, int> m = f(vec, s);
  for (const auto& pair : m) {
    std::cout << pair.first << ":" << pair.second << "-";
  }
  return 0;
}
```

Choose one or more options.

1. ⬜ 1:1-2:3-3:3:4:4-5:4-6:3-
2. ⬜ 2:3-4:4-6:3-
3. ⬜ 2:2-4:4-6:6-
4. ⬜ 3:2-4:4-3:6-
5. ⬜ 1:1-2:2-6:3-2:4-3:5-3:6-2:7-3:8-4:9-5:10-4:11-4:12-4:13-5:14-5:15-5:16-6:17-6:18-
6. ⬜ 1:1-2:3-3:3-4:4-5:4-6:3-

Ex 3 (1.5 points)

Analyze the following code snippet in C++. Indicate the possible output or outputs obtained by executing the program.

```
int main() {
    std::vector<int> vec = {1, 2, 3, 4, 5};
    auto lf = [&vec](long unsigned int index) -> int {
        if (index >= 0 && index < vec.size()) {
            int value = vec[index];
            return value;
        }
        std::cout << "Out" << " ";
        return -1;
    };
    std::cout << lf(2) << " ";
    std::cout << lf(5) << " ";
    std::cout << lf(0) << " ";
    return 0;
}
```

Choose one or more options:

1. ⬜ 2 Out -1 0
2. ⬜ 3 Out 1
3. ⬜ 3 -1 1
4. ⬜ 3 Out -1 1
5. ⬜ 2 -1 0
6. ⬜ 3 Out -1 Out 1 Out

Ex 4 (2.0 points)

Analyze the following code snippet in C++. When the main is executed, Indicate which of the following statements are correct.

```
class C {
private:
  ...
public:
  ...
};
```

```
void f1(C &e) { cout << "{f1}";}
void f2(C *e) { cout << "{f2}";}
void f3(shared_ptr<C> e) { cout << "{f3}";}

int main() {
  cout << "{01}"; C e1;
  cout << "{02}"; C *e2 = new C[3];
  cout << "{03}"; shared_ptr<C> e3 = shared_ptr<C> (new C);
  cout << "{04}"; shared_ptr<C> e4 = shared_ptr<C> (new C);
  cout << "{05}"; f1 (e1);
  cout << "{06}"; f2 (e2);
  cout << "{07}"; f3 (e3);
  cout << "{08}"; e1 = (std::move(e2[0]));
  cout << "{09}"; delete[] e2;
  cout << "{10}"; return 0;
}
```

Choose one or more options:

1. ⬜ Line number 1 includes one copy constructor.
2. ⬜ Line number 2 includes three standard constructors.
3. ⬜ Each one of lines number 3 and 4 includes one standard constructor and one copy assignment operator.
4. ⬜ Line number 5 includes one standard constructor.
5. ⬜ Line number 6 includes one standard constructor.
6. ⬜ Line number 7 includes one standard constructor.
7. ⬜ Line number 8 includes one move assignment operator.
8. ⬜ Line number 9 includes three destructors.
9. ⬜ Line number 10 includes three destructors.

Ex 5 (1.0 points)

Which of the following statements about templates in C++ are correct? Select all that apply.

1. ⬜ Function templates allow the same function to operate with different data types without code duplication.
2. ⬜ Class templates can create a class that works with any data type.
3. ⬜ Template specialization allows customizing the implementation of a template for specific data types.
4. ⬜ Templates can only be used with built-in data types like int, float, and double.
5. ⬜ Templates are instantiated at runtime, making the code slower due to type resolution.
6. ⬜ Template parameters can include types and non-type values such as integers or pointers.
7. ⬜ Templates improve code reusability and type safety without a runtime performance penalty.
8. ⬜ Templates in C++ are always compiled into separate template object files to be linked later.

Ex 6 (1.5 points)

Suppose to run the following program. Indicate the possible admissible outputs. Note that wrong answers imply a penalty in the final score.

```
sem_t *s;
int i, n;

static void *T (void *);

int main (int argc, char **argv) {
  char A='A', B='B', C='C';
  pthread_t th;
```

```
  s = (sem_t *) malloc (sizeof(sem_t));
  i = 0;
  n = 1;
  sem_init (s, 0, 1);

  setbuf(stdout, 0);

  pthread_create (&th, NULL, T, &A);
  pthread_create (&th, NULL, T, &B);
  pthread_create (&th, NULL, T, &C);

  pthread_exit(0);
}

static void *T (void *p) {
  char *tmp;
  char c;

  tmp = (char *) p;
  c = *tmp;
  pthread_detach (pthread_self ());

  while (1) {
    sem_wait (s);
    printf ("%c", c);
    i++;
    if (i>=n) {
      printf ("-");
      i = 0;
      n++;
      if (n>=4) {
      n = 1;
      }
    }
    sem_post (s);
  }

  return 0;
}
```

Choose one or more options.

1. ⬜ B-CA-BCA-B-CA-CBA-C-BA-CBA-C-AB-CBA-...
2. ⬜ B-AC-BAC-B-AC-ABC-A-BC-ABC-C-BA-BAC-C-AB-CAB-B-AC-ABC-B-...
3. ⬜ A-BA-CBA-B-AC-BAC-C-BA-CAB-...
4. ⬜ AA-AAA-BB-BBB-CC-CCC-...
5. ⬜ B-BB-BBB-B-BB-BBB-...
6. ⬜ CC-CCC-CCCC-AA-AAA-AAAA-...
7. ⬜ C-CC-CCC-A-AA-AAA-..
8. ⬜ CCC-CC-C-BBB-BB-B-AAA-AA-A-...
9. ⬜ ABC-ABC-ABC-...

Ex 7 (1.0 points)

Analyze the following code snippet. Which statements about using barriers in the following C code are correct? Note that wrong answers imply a penalty in the final score.

```
#define NUM_THREADS  16
#define NUM_CYCLEs  100

pthread_barrier_t barrier;

void* work(void* arg) {
  int id = *(int*)arg;
  For (int i=0; i<NUM_CYCLES; ++i) {
    sleep(id);
    printf("Thread %d is doing initial work.\n", id);
    pthread_barrier_wait(&barrier);
  }
  return NULL;
}

int main() {
  pthread_t threads[NUM_THREADS];
  int thread_ids[NUM_THREADS];
  pthread_barrier_init(&barrier, NULL, NUM_THREADS);
  for (int i=0; i<NUM_THREADS; ++i) {
    thread_ids[i] = i;
    pthread_create(&threads[i], NULL, work, &thread_ids[i]);
  }
  // LINE 1
  for (int i=0; i<NUM_THREADS; ++i) {
    pthread_join(threads[i], NULL);
  }
  // LINE 2
  return 0;
}
```

Select all that apply.

1. ⬜ The barrier ensures that all threads reach a synchronization point before any of them proceed with the post-barrier work.
2. ⬜ The threads are cyclic; thus, we must insert two barrier constructs to prevent a fast thread from performing more iterations than the slower ones.
3. ⬜ The `pthread_barrier_wait` function decreases the barrier count and blocks the thread until all threads have reached the barrier.
4. ⬜ If the number of threads exceeds the barrier count, the program will be deadlocked.
5. ⬜The barrier is initialized with a count equal to the number of threads to ensure proper synchronization.
6. ⬜In Line 1, the barrier must be destroyed to release resources.
7. ⬜ In Line 2, the barrier must be destroyed to release resources.
8. ⬜ The `pthread_barrier_init` function initializes the barrier with an optional attribute parameter and the number of threads, NUM_THREADS.

Ex 8 (1.0 points)

Which statements about thread pools in C and C++ programming are correct?

Select all that apply.

1. ⬜ Thread pools are designed to reduce the number of threads running in the more expensive sections of codes.

2. ⬜ Thread pools are designed to synchronize threads in specific positions of the code.
3. ⬜ A thread pool is a collection of pre-instantiated threads that can be reused to perform multiple tasks.
4. ⬜ Using a thread pool can help improve performance by reducing the overhead of creating and destroying threads for each task.
5. ⬜ Thread pools automatically manage the scheduling and execution of tasks; moreover, they ensure that tasks are executed in the order they are submitted.
6. ⬜ The C++ Standard Library provides built-in support for thread pools via the std::thread_pool class.
7. ⬜ In C, implementing a thread pool typically requires using the library POSIX threads (pthreads) or a similar threading library.
8. ⬜ Using a thread pool ensures that all tasks will be completed promptly without risk of starvation.

Ex 9 (1.0 points)

Which of the following statements are true regarding `std::thread` and `std::async` in C++?
Select all that apply.

1. ⬜ `std::thread` is used to create a new thread of execution.
2. ⬜ `std::async` guarantees that the task runs on a separate thread.
3. ⬜ `std::thread` and `std::async` cannot be used together in the same program.
4. ⬜ `std::async` can automatically manage the synchronization of the task's result.
5. ⬜ `std::thread` automatically manages the execution and synchronization of the task.
6. ⬜ `std::async` creates a task that runs on the main thread.
7. ⬜ `std::thread` requires explicit management of the thread's lifetime, while `std::async` automatically manages the execution and synchronization of the task.
8. ⬜ When we use `std::thread` we cannot capture a value the thread returns.

Ex 10 (1.0 points)

The following code snippet demonstrates the use of futures and promises in C++.
Which of the following statements about the code and futures and promises in general are correct?

```
void doWork(std::promise<int>& promiseObj) {
  std::this_thread::sleep_for(std::chrono::seconds(2));
  promiseObj.set_value(10);
}

int main() {
  std::promise<int> promiseObj;
  std::future<int> futureObj = promiseObj.get_future();
  std::thread t(doWork, promiseObj);
  int result1 = futureObj.get();
  int result2 = futureObj.get();
  std::cout << "Result from future: " << result1 << " " << result2 << std::endl;
  t.join();
  return 0;
}
```

Select all that apply.

1. ⬜ A `std::promise` object is used to set a value or an exception that can be retrieved by a `std::future object`.
2. ⬜ A `std::future` object can retrieve the value set by a `std::promise`.
3. ⬜ The `std::promise` and `std::future` are used exclusively with `std::async`.
4. ⬜ The `std::promise` transfers the ownership of a `std::thread` to another thread.
5. ⬜ As we get the future with `futureObj.get()`, we do not have to perform a join on the thread, i.e., we do not have to execute `t.join()`.

6. ⬜ A `std::future` object blocks the calling thread until the value is available or an exception is set.
7. ⬜ The line `int result2 = futureObj.get();` will correctly retrieve the value set by the promise a second time.
8. ⬜ The promise object is passed correctly to the thread function `doWork`.
9. ⬜ The promise object must be passed by reference to the `doWork` function, with `std::ref(promiseObj)`.

Ex 11 (1.0 points)

Analyze the following code snippet. Which one of the following questions is correct?

```
int main() {
  int fd1 = open("file1.txt", O_RDONLY);
  int fd2 = open("file2.txt", O_RDONLY);
  int fd3 = open("file3.txt", O_RDONLY);
  if (fd1 == -1 || fd2 == -1 || fd3 == -1) {
     perror("Failed to open files");
     return 1;
  }
  fd_set readfds;
  FD_ZERO(&readfds);
  FD_SET(fd1, &readfds);
  FD_SET(fd2, &readfds);
  // Line 1

  int maxfd = ...  // Line 2
  struct timeval tv;
  tv.tv_sec = 5;
  tv.tv_usec = 0;

  int result = select(maxfd, &readfds, NULL, NULL, &tv);
  if (result == -1) {
    perror("select error");
  } else if (result == 0) {
    printf("Timeout occurred! No data after 5 seconds.\n");
  } else {
    if (FD_ISSET(fd1, &readfds)) {
      printf("fd1 is ready for reading\n");
    }
    if (FD_ISSET(fd2, &readfds)) {
      printf("fd2 is ready for reading\n");
    }
    if (FD_ISSET(fd3, &readfds)) {
      printf("fd3 is ready for reading\n");
    }
 }
 close(fd1);
  close(fd2);
  return 0;
}
```

Select all that apply.

1. ⬜ The `fd_set` structure specifies the file descriptors you want to monitor. The set of monitor descriptors can be monitored: the reading, writing, and execution sets.
2. ⬜ In Line 2, we must compute the maximum value among fd1, fd2, and fd3 and add 1 to this value.
3. ⬜ In Line 2, we must compute the maximum value among fd1, fd2, and fd3.

4. ⬜ Setting `tv.tv_sec` and `tv.tv_usec` to zero causes `select` to return immediately, effectively polling the file descriptors without blocking. This can be useful for non-blocking checks.
5. ⬜ In Line 1, we must insert `FD_ISSET(fd3, &readfds)`.
6. ⬜ In Line 1, we must insert `FD_SET(fd3, &readfds)`.
7. ⬜ The descriptors `fd1`, `fd2`, and `fd3` can be equal to -1.
8. ⬜ You can add `fd_set` variables for write and exception file descriptors and pass them to select. For example, `fd_set writefds; FD_ZERO(&writefds); FD_SET(fd1, &writefds); select(maxfd + 1, &readfds, &writefds, &tv);`.
9. ⬜ To maintain a responsive program, we must Implement a loop to continuously call `select` and handle ready file descriptors. This approach allows the program to respond to file descriptor changes dynamically.

Ex 12 (1.0 points)

Analyze the following two code snippets. Which one of the following questions is correct?

```
int pipefd[2];
if (pipe(pipefd) == -1) {
  fprintf (stderr, "Error on pipe.\n");
  exit (1);
}

if (mkfifo("/tmp/myfifo", 0666) == -1) {
  fprintf (stderr, "Error on mkfifo.\n");
  exit (1);
}
```

Select all that apply.

1. ⬜ A pipe is a unidirectional communication channel that can be used between related processes (such as parent and child processes).
2. ⬜ A FIFO is similar to a pipe but can communicate between unrelated processes.
3. ⬜ The function `pipe` returns two file descriptors: one for reading (`pipefd[1]`) and one for writing (`pipefd[0]`).
4. ⬜ After creating a FIFO, we must open it FIFO using the open function, specifying the path to the FIFO and the desired access mode (O_RDONLY for reading, O_WRONLY for writing, or O_RDWR for both).
5. ⬜ Function `mkfifo` creates a directory at the specified path.
6. ⬜ The purpose of the 0666 argument in the `mkfifo` function call is to set the FIFO to be readable and writable by the owner only.
7. ⬜ The purpose of the 0666 argument in the `mkfifo` function call is to set the FIFO to be readable and writable by the owner, group, and others.
8. ⬜ The function exit(1) exits the current function and returns to the caller.